МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«Национальный исследовательский университет ИТМО»

ФАКУЛЬТЕТ ПРОГРАММНОЙ ИНЖЕНЕРИИ И
КОМПЬЮТЕРНОЙ ТЕХНИКИ

КУРСОВАЯ РАБОТА

по дисциплине

«ИНФОРМАЦИОННЫЕ СИСТЕМЫ»

Система менеджмента персонала

Выполнили:
Рязанов Никита Сергеевич,
Софьин Вячеслав Евгеньевич,
студенты группы P3307

Проверил:
Демурчян Владимир Варламович

г. Санкт-Петербург, 2026

Содержание

Задание 3
Инфологическая модель 4
Даталогическая модель 6
Реализация в PostgreSQL 7
Обеспечение целостности данных 8
    Триггеры 8
Функции и процедуры 9
Индексы 10
Вывод 12

Задание

1. Сформировать ER-модель базы данных (на основе описаний предметной области и прецедентов из предыдущего этапа). ER-модель должна:
   - включать в себя не менее 10 сущностей;
   - содержать хотя бы одно отношение вида «многие-ко-многим».
2. Согласовать ER-модель с преподавателем. На основе ER-модели построить даталогическую модель.
3. Реализовать даталогическую модель в реляционной СУБД PostgreSQL.
4. Обеспечить целостность данных при помощи средств языка DDL и триггеров.
5. Реализовать скрипты для создания, удаления базы данных, заполнения базы тестовыми данными.
6. Предложить pl/pgsql-функции и процедуры, для выполнения критически важных запросов (которые потребуются при последующей реализации прецедентов).
7. Создать индексы на основе анализа использования базы данных в контексте описанных на первом этапе прецедентов. Обосновать полезность созданных индексов для реализации представленных на первом этапе бизнес-процессов.
8. Составить отчет.

Инфологическая модель

ER-модель предметной области содержит 20 сущностей (Рис. 1).

Основные связи между сущностями:

- компания включает множество пользователей и команд (1:N);
- команда возглавляется одним тимлидом (1:1), включает множество сотрудников (1:N) и публикует множество вакансий (1:N);
- вакансия расположена в одном городе (N:1);
- кандидат подает множество откликов, на вакансию приходит множество откликов; отклик ссылается на одно резюме и одну анкету;
- отклик получает множество оценок HR (1:N) и одну агрегированную оценку (1:1);
- интервью связывает кандидата, тимлида и вакансию; по интервью оставляется не более одного отзыва (1:1);
- задача имеет множество тестов (1:N) и одного автора (N:1);
- посылка относится к одному отправителю, контесту, задаче и языку программирования (N:1);
- пользователь имеет множество отчетов о GitHub-активности (1:N).

Отношения вида «многие-ко-многим»:

- `Vacancy` – `Skill`: вакансия требует набор навыков, навык востребован во многих вакансиях;
- `Form` – `Skill`: кандидат указывает в анкете навыки, которыми владеет;
- `Contest` – `Task`: контест состоит из нескольких задач, задача может использоваться в разных контестах;
- `Vacancy` – `Contest`: к вакансии привязываются отборочные контесты, один контест может использоваться для нескольких вакансий;
- `Contest` – `Person`: участники контеста.

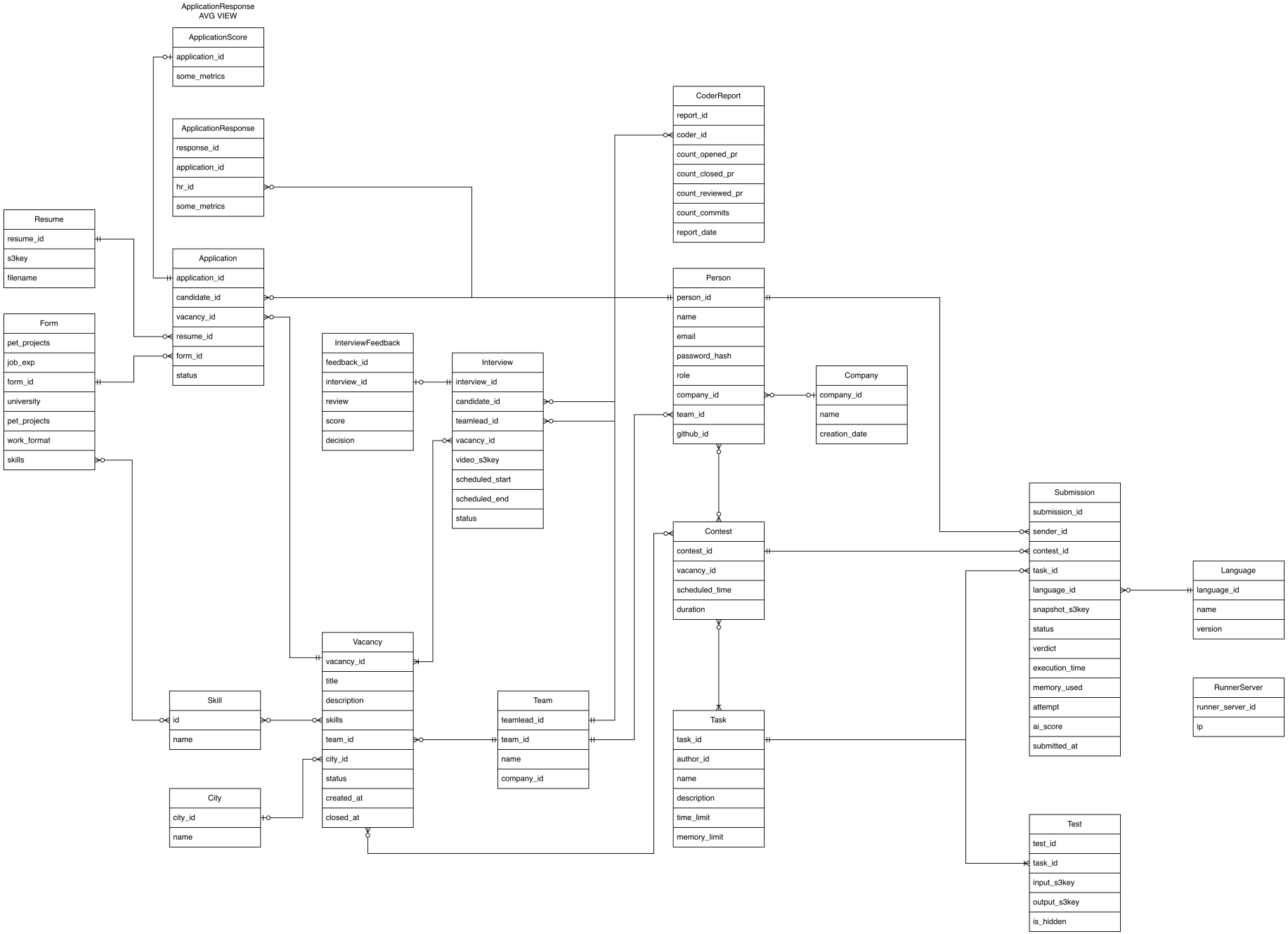


Рис. 1. Инфологическая модель (ER-диаграмма)

Даталогическая модель

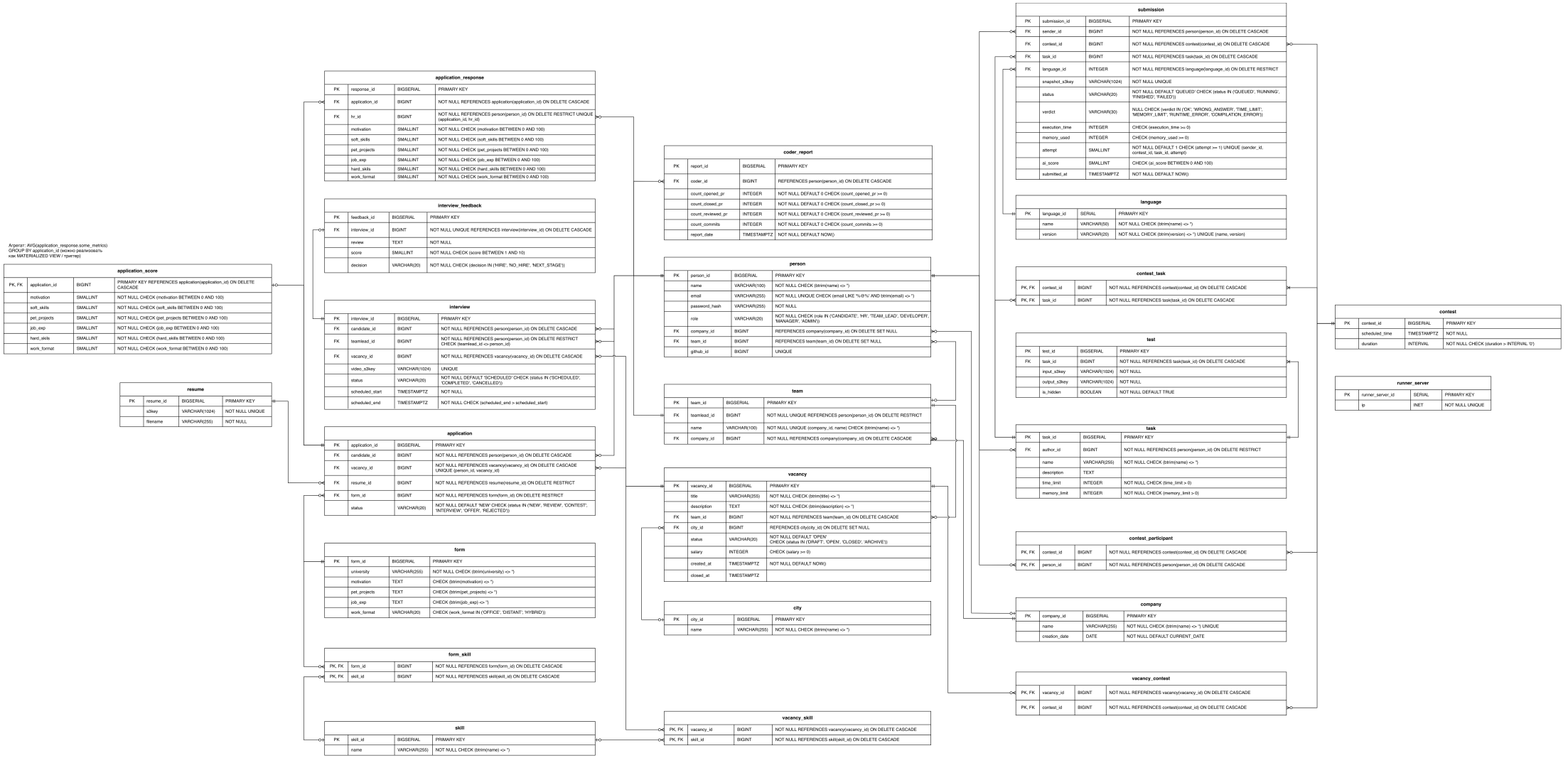


Рис. 2. Даталогическая модель

Реализация в PostgreSQL

Даталогическая модель реализована в СУБД PostgreSQL.

Скрипты расположены в репозитории проекта:
https://github.com/safarislava/itmo-team

Скрипт	**Назначение**
`create.sql`	Создание таблиц и ограничений целостности
`func.sql`	Функции, процедуры, триггеры и задание планировщика `pg_cron`
`index.sql`	Создание индексов
`fill.sql`	Заполнение базы тестовыми данными
`truncate.sql`	Очистка всех таблиц со сбросом последовательностей
`drop.sql`	Удаление всех таблиц

Таблица 1. Скрипты базы данных

База данных разворачивается последовательным выполнением скриптов.

Для работы скрипта `func.sql` требуется расширение `pg_cron`, которое должно быть подключено на сервере PostgreSQL.

Обеспечение целостности данных

Триггеры

Ограничения, которые невозможно выразить декларативно (зависящие от данных других таблиц), реализованы триггерами:

Триггер таблица, событие	**Назначение**
`trg_check_team_teamlead_id` `team`, `BEFORE INSERT OR UPDATE`	Проверяет, что руководитель команды имеет роль `TEAM_LEAD`.
`trg_check_application_response_hr_id` `application_response`, `BEFORE INSERT OR UPDATE`	Проверяет, что оценку отклику выставляет пользователь с ролью `HR`.
`trg_check_application_candidate_id` `application`, `BEFORE INSERT OR UPDATE`	Проверяет, что отклик подает пользователь с ролью `CANDIDATE`.
`trg_check_interview_candidate_id` `interview`, `BEFORE INSERT OR UPDATE`	Проверяет, что интервью назначается пользователю с ролью `CANDIDATE`.
`trg_check_application_for_open_vacancy` `application`, `BEFORE INSERT`	Запрещает отклик на вакансию, не находящуюся в статусе `OPEN` (UC-01).
`trg_increment_attempt_counter` `submission`, `BEFORE INSERT`	Автоматически вычисляет номер попытки как следующий за максимальным для тройки «отправитель – контест – задача» (UC-02).

`trg_refresh_application_score`
`application_response`,
`AFTER INSERT OR UPDATE OR DELETE`
	Пересчитывает агрегированные оценки откликов в `application_score` как средние значения оценок HR (UC-04).

Таблица 2. Триггеры

Функции и процедуры

Для критически важных запросов, необходимых при реализации прецедентов, на языке PL/pgSQL реализованы функции и процедуры:

Функция / процедура	**Назначение**
`match_vacancies` функция	Поиск открытых вакансий, требующих все указанные навыки, с необязательными фильтрами по городу и зарплате (UC-05).
`archive_vacancies` процедура	Перевод вакансий, закрытых более 6 месяцев назад, в статус `ARCHIVE`. Запускается ежедневно в 00:00 через `pg_cron`.
`close_vacancy_with_rejections` функция	Закрытие вакансии и отклонение всех откликов на нее, кроме получивших оффер. Возвращает идентификаторы отклоненных откликов для уведомления кандидатов (UC-11, UC-18).
`get_candidate_leaderboard` функция	Рейтинг кандидатов на вакансию: рекомендательный балл — взвешенная сумма агрегированных оценок HR с весами, заданными тимлидом (UC-09, UC-16).
`schedule_interview` процедура	Назначение интервью по отклику: проверка, что у тимлида нет пересекающихся интервью, создание интервью и перевод отклика в статус `INTERVIEW` (UC-10).

Таблица 3. Функции и процедуры

Индексы

На основе анализа запросов прецедентов созданы индексы, приведенные в таблице 4. Все индексы – B-tree, так как используются для поиска по равенству и соединений.

Индекс столбцы	**Обоснование**
`idx_vacancy_city_id` `vacancy (city_id)`	Фильтр по городу в функции `match_vacancies` при поиске вакансий (UC-05). Проверка ссылок при удалении города из справочника (UC-21).
`idx_vacancy_skill_skill_id` `vacancy_skill (skill_id, vacancy_id)`	Первичный ключ `(vacancy_id, skill_id)` обслуживает выборку навыков вакансии, но не обратный поиск вакансий по навыку при фильтрации по стеку (UC-05).
`idx_vacancy_team_id` `vacancy (team_id)`	Выборка вакансий команды тимлида: просмотр кандидатов в команду (UC-09), открытие вакансий (UC-12), аналитика найма по подразделению (UC-15).
`idx_form_skill_skill_id` `form_skill (skill_id, form_id)`	Поиск кандидатов по стеку технологий тимлидом (UC-09): по навыку находятся анкеты, по ним – отклики.
`idx_application_vacancy_id` `application (vacancy_id)`	Выборка откликов на вакансию в `get_candidate_leaderboard` (UC-09, UC-16) и при оценке анкет HR (UC-04), массовое обновление статусов в `close_vacancy_with_rejections` (UC-11).
`idx_interview_person_id` `interview (candidate_id)`	Выборка интервью кандидата при просмотре статусов (UC-06) и подключении к сессии лайв-кодинга (UC-07).
`idx_vacancy_contest_vacancy_id` `vacancy_contest (vacancy_id)`	Поиск отборочного контеста вакансии после подачи отклика (UC-01) для предоставления кандидату доступа к контесту.

Индекс столбцы	**Обоснование**
`idx_contest_task_contest_id` `contest_task (contest_id)`	Выборка задач контеста при его прохождении кандидатом (UC-02).
`idx_test_task_id` `test (task_id)`	Выборка тестов задачи подсистемой проверки при тестировании каждой посылки (UC-03) – наиболее частый запрос подсистемы тестирования. Управление тестами в банке задач (UC-13).
`idx_submission_task_id` `submission (task_id)`	Статистика решений по задаче в банке задач (UC-13), проверка ссылок при удалении задачи.
`idx_submission_contest_id` `submission (contest_id)`	Сводные результаты контеста при оценке кандидатов HR (UC-04) и тимлидом (UC-09, UC-11).

Таблица 4. Индексы

Вывод

В ходе второго этапа курсовой работы на основе описания предметной области и прецедентов первого этапа построены инфологическая модель и соответствующая ей даталогическая модель.

Модель реализована в СУБД PostgreSQL. Подготовлены скрипты создания, удаления, очистки и заполнения базы тестовыми данными. Для критически важных операций реализованы функции и процедуры на PL/pgSQL, которые будут использованы на уровне хранения информационной системы на следующем этапе. Были созданы индексы, ускоряющие частые запросы основных прецедентов.